



DOSW COMPANY



ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 2

GRUPO 1



QUIZ #1



Ingresen al Código que se va a generar con Quizizz

Solo van a tener un intento

15 preguntas de opcion multiple – tienen 10 minutos

Los estudiantes que estan exonerados pueden presentarlo ya que les sirve para afianzar conocimientos



ESCUELA COLOMBIANA DE INGENIERÍA

¿QUE VAMOS A VER HOY?

NUEVOS TEMAS

- SOLID
- PATRONES DE DISEÑO
- DIAGRAMAS DE CLASES
- MAVEN

INICIO DEL BOOTCAMP



DOSW COMPANY



BITACORA

- Semana a semana Se ira construyendo con los temas que veamos en clase
- Incluso los ejercicios que se realicen completamente en clase deben quedar codificados en la bitácora.
- En algunos casos, se dejarán ejercicios sencillos para completar en casa, como parte natural del proceso de práctica.
- Su Bitacora sera un repositorio en GitHub





EJERCICIOS DE REPASO

EJERCICIO # 1:

Dada una lista de numeros enteros, necesitamos obtener una nueva lista solo con los numeros pares mayores a diez

Datos de Entrada:

List<Integer> numbers = List.of(3,8,10,12,15,18,20)

Resultado:

List<Integer>

```
List<Integer> result = numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .filter(n -> n > 10)  
    .toList();
```

```
List<Integer> resultado = numbers.stream()  
    .filter(n -> n % 2 == 0 && n > 10)  
    .toList();
```



EJERCICIOS DE REPASO

EJERCICIO # 2:

Dada una lista de palabras, se requiere:

- Filtrar las palabras que tengan mas de 4 caracteres
- Convertirlas en Mayusculas
- Ordenarlas alfabeticamente
- Obtener la cantidad total de palabras resultantes

Datos de Entrada:

List<String> words =

List.of("java","stream","api","functional","code","git")

```
List<String> processed = words.stream()
    .filter(w -> w.length() > 4)
    .map(String::toUpperCase)
    .sorted()
    .toList();

long count = processed.stream()
    .count();
```

```
long cantidad = words.stream()
    .filter(w -> w.length() > 4)
    .map(String::toUpperCase)
    .sorted()
    .peek(System.out::println)
    .count();
```



EJERCICIOS DE REPASO

EJERCICIO # 3:

Dada una lista de usuarios con los atributos:

- id
- name
- age
- active

Filtra unicamente los usuarios activos

Obten una lista con los nombres en mayuscula

Ordena alfabeticamente el resultado

Datos de Entrada: `users = List<Users>`

Resultado: `List<String>`

```
List<String> result = users.stream()  
    .filter(User::isActive)  
    .map(User::getName)  
    .map(String::toUpperCase)  
    .sorted()  
    .toList();
```

```
List<String> resultado = users.stream()  
    .filter(Usuario::active)  
    .map(u -> u.name().toUpperCase())  
    .sorted()  
    .toList();
```




EJERCICIOS DE REPASO

EJERCICIO # 4:

Dado un listado de Usuarios y utilizando los mismo atributos anteriores, filtrar las personas mayores de edad y obtener sus nombres.

```
List<Usuario> users = List.of(  
    new Usuario(1, "carlos", 17, true),  
    new Usuario(2, "ana", 30, false),  
    new Usuario(3, "miguel", 15, true),  
    new Usuario(4, "beatriz", 28, false),  
    new Usuario(5, "juan", 35, true)  
);  
  
List<String> resultado = users.stream()  
    .filter(u -> u.age() >= 18)  
    .map(Usuario::name)  
    .toList();
```




EJERCICIOS DE REPASO

EJERCICIO # 5:

Dada una lista de transacciones bancarias representadas por objetos:

```
class Transaction {  
    String id;  
    double amount;  
    boolean approved;  
}
```

Se requiere procesar la lista usando Streams para:

- Usar peek para ver cada transacción procesada (Utilizar System.out.println para ver la transacción)
- Verificar si existe al menos una transacción no aprobada
- Retornar true o false indicando si el lote de transacciones es válido.

```
List<Transaction> transactions = List.of(  
    new Transaction("TX-001", 120.50, true),  
    new Transaction("TX-002", 350.00, true),  
    new Transaction("TX-003", 90.25, false),  
    new Transaction("TX-004", 500.00, true)  
);
```

```
boolean hasUnapprovedTransaction = transactions.stream()  
    .peek(System.out::println)  
    .anyMatch(transaction -> !transaction.isApproved());  
  
return !hasUnapprovedTransaction;
```



TAREA - #1

- Adjunten los 5 ejercicios anteriores en su bitacora
- Desarrollen el taller #2 de programacion funcional en su bitacora

NOTA: Ahi encontraran la estructura de paquetes para la misma



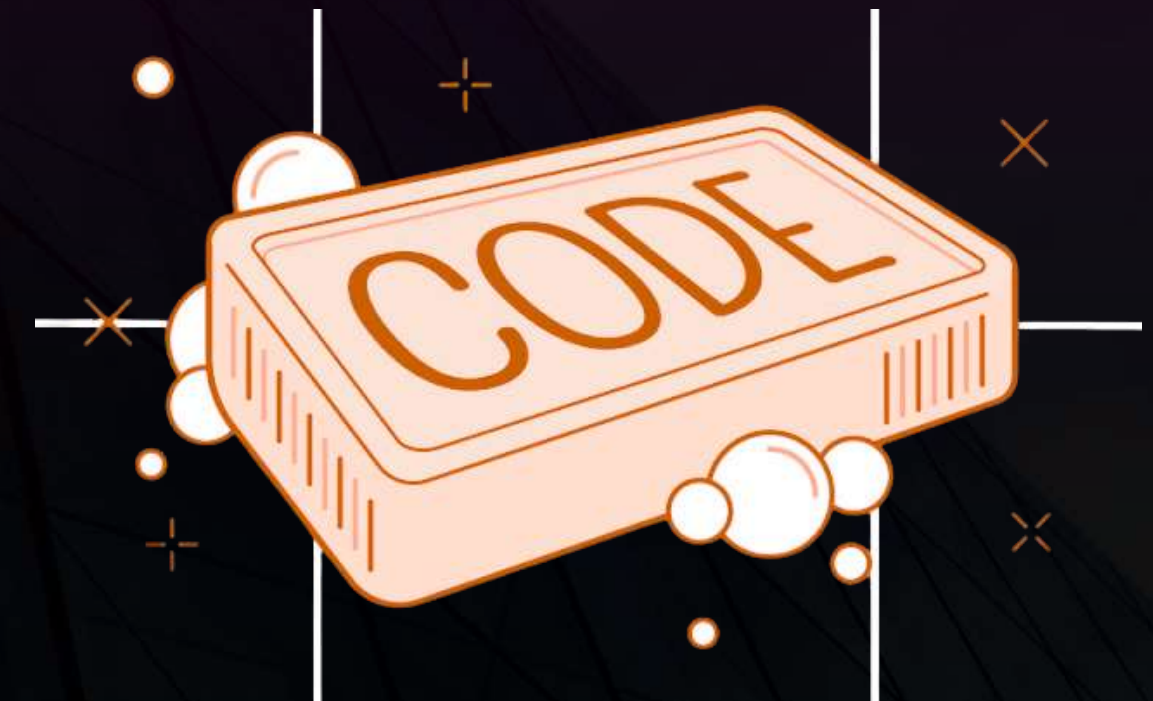
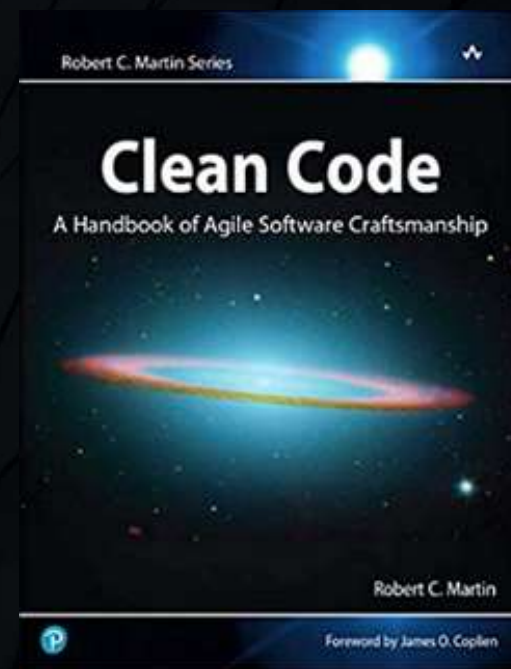


DOSW COMPANY

S.O.L.I.D

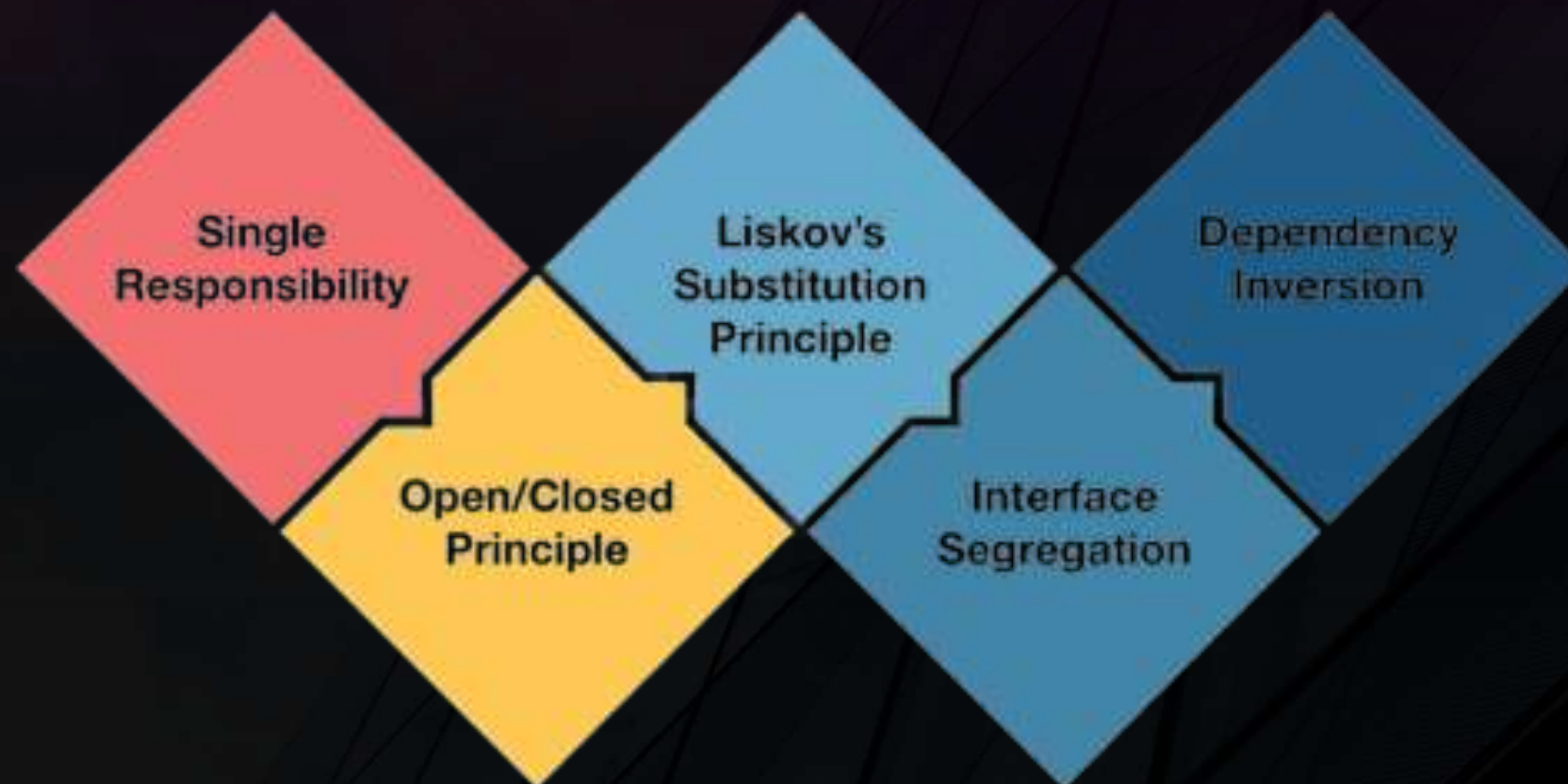
CLEAN CODE

- Es una Filosofía utilizada en el desarrollo de software cuyo objetivo es hacer mas facil la lectura y escritura de codigo. Se basa en la aplicacion de tecnicas sencillas con las que generamos un codigo claro e intuitivo que es mas facil de modificar



- Robert J.Martin (tambien conocido como el tio Bob), Introdujo el concepto de SOLID, pero el acronimo fue introducido por Michael Feathers.

¿QUE ES SOLID?



Conjunto de reglas y mejores practicas a seguir al diseñar una estructura de clase

Estos principios nos ayudan a comprender la necesidad de ciertos patrones de diseño y arquitectura de software en general

Todos tienen el mismo proposito: Crear codigo mantenible mas facil y rapido, permite añadir nuevas funcionalidades de forma sencilla, favorece una mayor reusabilidad y calidad del codigo



DOSW COMPANY



S

Single responsibility principle

O

Open/closed principle

L

Liskov substitution principle

I

Interface segregation principle

D

Dependency inversion principle



S: SINGLE RESPONSIBILITY

Establece que una clase, componente o servicio debe ser responsable de una sola cosa (desacople)

Si una clase realiza multiples responsabilidades, sera mas dificil de mantener y modificar sin afectar otras partes del sistema

CONSEJO DE APLICACION



Reune las cosas que cambian por las mismas razones y separa aquellas que cambian por razones diferentes

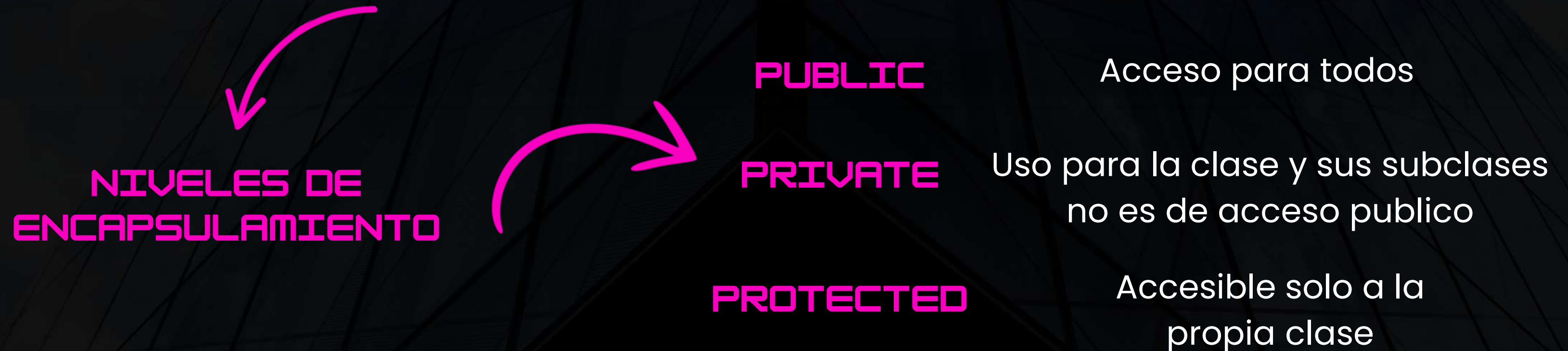
Utiliza el encapsulamento para proteger los datos internos de un objeto.



S: SINGLE RESPONSIBILITY

ENCAPSULAMIENTO

Consiste en ocultar los detalles internos de una clase y permitir el acceso a su información únicamente a través de operaciones definidas para ese objeto



S: SINGLE RESPONSIBILITY

¿Por que este codigo viola el principio?

```
class Coche {  
    String marca;  
  
    Coche(String marca){ this.marca = marca; }  
  
    String getMarcaCoche(){ return marca; }  
  
    void guardarCocheDB(Coche coche){ ... }  
}
```



¿Como podria mejorar?

```
class Coche {  
    String marca;  
  
    Coche(String marca){ this.marca = marca; }  
  
    String getMarcaCoche(){ return marca; }  
}  
  
class CocheDB{  
    void guardarCocheDB(Coche coche){ ... }  
    void eliminarCocheDB(Coche coche){ ... }  
}
```





SINGLE RESPONSIBILITY FALLA SI...



En una clase estan involucradas dos capas de arquitectura



Si contiene demasiados metodos publicos



Es dificil de Probar (pruebas unitarias y granularidad)



Una nueva funcionalidad la afecta



O: OPEN/CLOSED

Establece que las entidades de software (clases, modulos y funciones) deberian estar abiertos para su extension pero cerrados para su modificacion

Deberiamos agregar nuevas funciones sin tocar el codigo existente, evitando el riesgo de crear errores potenciales

CONSEJO DE APLICACION



Se suele resolver utilizando POLIMORFISMO al no obligar a la clase principal a saber como realizar la operacion, se le delega a los objetos que lo utiliza

O: OPEN/CLOSED

POLIMORFISMO

Capacidad de los objetos de una clase en ofrecer respuestas distintas e independientes en funcion de sus parametros durante su ejecucion

OVERLOADING

Un mismo metodo con firmas distintas

```
public class Calculadora {  
  
    // Método para sumar dos enteros  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
  
    // Sobrecarga: método para sumar tres enteros  
    public int sumar(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Sobrecarga: método para sumar dos doubles  
    public double sumar(double a, double b) {  
        return a + b;  
    }  
}
```

OVERRIDING

Un mismo metodo sobreescrito

```
// Clase base  
public class Animal {  
    public void hacerSonido() {  
        System.out.println("El animal hace un sonido");  
    }  
}  
  
// Subclase  
public class Perro extends Animal {  
    @Override  
    public void hacerSonido() {  
        System.out.println("El perro ladra: ¡Guau guau!");  
    }  
}  
  
// Subclase  
public class Gato extends Animal {  
    @Override  
    public void hacerSonido() {  
        System.out.println("El gato maúlla: ¡Miau!");  
    }  
}
```

CLASES
ABSTRACTAS
extends

INTERFACES
implements

una interface proporciona un contrato/mecanismo de encapsulación de los protocolos de los métodos sin forzar al usuario a utilizar la herencia.

O: OPEN/CLOSED

¿Por que este codigo viola el principio si añadimos un nuevo coche de otra marca?

```
public static void main(String[] args) {  
    Coche[] arrayCoches = {  
        new Coche("Renault"),  
        new Coche("Audi")  
    };  
    imprimirPrecioMedioCoche(arrayCoches);  
}  
  
public static void imprimirPrecioMedioCoche(Coche[] arrayCoches){  
    for (Coche coche : arrayCoches) {  
        if(coche.marca.equals("Renault")) System.out.println(18000);  
        if(coche.marca.equals("Audi")) System.out.println(25000);  
    }  
}
```



Tendriamos que modificar el metodo que hemos creado anteriormente

```
Coche[] arrayCoches = {  
    new Coche("Renault"),  
    new Coche("Audi"),  
    new Coche("Mercedes")  
};
```

```
public static void imprimirPrecioMedioCoche(Coche[] arrayCoches){  
    for (Coche coche : arrayCoches) {  
        if(coche.marca.equals("Renault")) System.out.println(18000);  
        if(coche.marca.equals("Audi")) System.out.println(25000);  
        if(coche.marca.equals("Mercedes")) System.out.println(27000);  
    }  
}
```



O: OPEN/CLOSED

¿Como podria mejorar?

```
abstract class Coche {  
    // ...  
    abstract int precioMedioCoche();  
}  
  
class Renault extends Coche {  
    @Override  
    int precioMedioCoche() { return 18000; }  
}  
  
class Audi extends Coche {  
    @Override  
    int precioMedioCoche() { return 25000; }  
}  
  
class Mercedes extends Coche {  
    @Override  
    int precioMedioCoche() { return 27000; }  
}  
  
public static void main(String[] args) {  
  
    Coche[] arrayCoches = {  
        new Renault(),  
        new Audi(),  
        new Mercedes()  
    };  
  
    imprimirPrecioMedioCoche(arrayCoches);  
}  
  
public static void imprimirPrecioMedioCoche(Coche[] arrayCoches){  
    for (Coche coche : arrayCoches) {  
        System.out.println(coche.precioMedioCoche());  
    }  
}
```



Cada coche extiende la clase abstracta Coche e implementa el metodo abstracto precioMedioCoche()



Cada coche tiene su propia implementacion del metodo, por lo que el metodo imprimirPrecioMedioCoche itera el array y solo llama al metodo



Si añadimos un nuevo coche, precioMedioCoche no tendra que ser modificado



OPEN/CLOSED FALLA SI...



Se detecta si es una clase que se debe modificar muy a menudo y estos cambios afectan lo ya modificado



Uso excesivo de if/else o switch en vez de polimorfismo



Codigo duplicado con variaciones minimas



Si modificar una funcionalidad obliga a tocar muchas clases o metodos no relacionados con la nueva extension, el sistema no esta bien cerrado a modificaciones



L: LISKOV SUBSTITUTION

Las clases derivadas (subclase) deben poder sustituirse por sus clases base (superclase)

Al extender una clase no se altera el comportamiento de la clase padre

CONSEJO DE APLICACION



Con INMUTABILIDAD se puede respetar este principio, ya que con ella podremos utilizar herencia de forma correcta

PATO DE GOMA VS PATO ANIMAL



L: LISKOV SUBSTITUTION

INMUTABILIDAD

Una vez que se ha creado un objeto no puede volver a modificarse

Mejor uso de memoria ya que todo su estado es final

Seguridad en multiples hilos de ejecucion

```
final class Punto {  
    private final int x;  
    private final int y;  
  
    public Punto(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int getX() {  
        return x;  
    }  
  
    public int getY() {  
        return y;  
    }  
  
    // No hay setters  
}
```

L: LISKOV SUBSTITUTION

¿Por que este codigo viola el principio?

```
// ...
public static void imprimirNumAsientos(Coche[] arrayCoches){
    for (Coche coche : arrayCoches) {
        if(coche instanceof Renault)
            System.out.println(numAsientosRenault(coche));
        if(coche instanceof Audi)
            System.out.println(numAsientosAudi(coche));
        if(coche instanceof Mercedes)
            System.out.println(numAsientosMercedes(coche));
    }
}
imprimirNumAsientos(arrayCoches);
```



Y si agregamos un nuevo tipo de Coche

```
// ...
Coche[] arrayCoches = {
    new Renault(),
    new Audi(),
    new Mercedes(),
    new Ford()
};

public static void imprimirNumAsientos(Coche[] arrayCoches){
    for (Coche coche : arrayCoches) {
        if(coche instanceof Renault)
            System.out.println(numAsientosRenault(coche));
        if(coche instanceof Audi)
            System.out.println(numAsientosAudi(coche));
        if(coche instanceof Mercedes)
            System.out.println(numAsientosMercedes(coche));
        if(coche instanceof Ford)
            System.out.println(numAsientosFord(coche));
    }
}
imprimirNumAsientos(arrayCoches);
```


L: LISKOV SUBSTITUTION

¿Como podria mejorar?

```
public static void imprimirNumAsientos(Coche[] arrayCoches){  
    for (Coche coche : arrayCoches) {  
        System.out.println(coche.numAsientos());  
    }  
}  
  
imprimirNumAsientos(arrayCoches);
```

Al metodo no le importa el tipo de la clase



Si la super clase Coche tiene un metodo que acepta un parametro del tipo de la super clase entonces su subclase deberia aceptar como argumento un tipo de la super clase o un tipo de la subclase



Si la superclase devuelve un tipo de ella misma (Coche) entonces su subclase deberia devolver un tipo de la superclase o un tipo de la subclase

L: LISKOV SUBSTITUTION

¿Como podria mejorar?

```
abstract class Coche {  
    // ...  
    abstract int numAsientos();  
}
```

```
class Renault extends Coche {  
    // ...  
    @Override  
    int numAsientos() {  
        return 5;  
    }  
    // ...  
}
```



La superclase debe definir el nuevo metodo y las subclases deben implementar dicho metodo



El metodo imprimirNumAsientos() no necesita saber con que tipo de coche se va a realizar su logica, simplemente llama al metodo numAsientos() del tipo Coche, ya que por contrato una subclase debe implementarlo



LSKOV SUBSTITUTION FALLA SI...

- ✗ Un metodo sobreescrito no hace nada o lanza una excepcion
- ✗ Si las pruebas padre no funcionan para la clase hija
- ✗ Si el método en la subclase exige más condiciones de entrada que el de la clase padre.
- ✗ Si el hijo no respeta las reglas de uso, garantías y contratos del padre, lo que genera errores, comportamientos inesperados o la necesidad de tratar al hijo de forma distinta.



I: INTERFACE SEGREGATION

Este principio establece que los clientes no deberian verse forzados a depender de interfaces que no usan

Cuando un cliente depende de una clase que implementa una interfaz cuya funcionalidad este cliente no usa pero otros clientes si, este cliente esta siendo afectado por los cambios que fuercen otros clientes en dicha interfaz

CONSEJO DE APLICACION



Es necesario definir una interfaz para cada tipo de cliente y no una general que agrupe metodos sin uso

Cada clase que implemente una interfaz debe utilizar TODOS los metodos

Una interfaz ayuda a desacoplar modulos entre si

I: INTERFACE SEGREGATION

¿Porque este codigo esta bien pero si implementamos Pinguinos se esta violando el principio?



```
interface IAve {  
    void volar();  
    void comer();  
}  
  
class Loro implements IAve{  
  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //..  
    }  
}  
  
class Tucan implements IAve{  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //..  
    }  
}
```



```
interface IAve {  
    void volar();  
    void comer();  
    void nadar();  
}  
  
class Loro implements IAve{  
  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //...  
    }  
  
    @Override  
    public void nadar() {  
        //...  
    }  
}  
  
class Pinguino implements IAve{  
  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //...  
    }  
  
    @Override  
    public void nadar() {  
        //...  
    }  
}
```

I: INTERFACE SEGREGATION

¿Porque este codigo esta bien pero si implementamos Pingüinos se esta violando el principio?

```
interface IAve {  
    void volar();  
    void comer();  
    void nadar();  
}  
  
class Loro implements IAve {  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //...  
    }  
  
    @Override  
    public void nadar() {  
        //...  
    }  
}  
  
class Pinguino implements IAve {  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //...  
    }  
  
    @Override  
    public void nadar() {  
        //...  
    }  
}
```



El problema es que el loro no nada y el pinguino no vuela



Si quisieramos añadir otro metodo a la interfaz IAve tendríamos que recorrer cada una de las clases que la implementa e ir añadiendo la implementacion de dicho metodo en todas ellas



Los clientes no tienen porque depender de metodos que no usan

I: INTERFACE SEGREGATION

¿Como se podria arreglar esto?

```
interface IAve {  
    void comer();  
}  
interface IAveVoladora {  
    void volar();  
}  
  
interface IAveNadadora {  
    void nadar();  
}
```



Cada clase implementa las interfaces de la que realmente necesita implementar sus metodos



Esto nos ahorra tiempo y nos permite añadir nuevas funcionalidades

```
class Loro implements IAve, IAveVoladora{  
  
    @Override  
    public void volar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //...  
    }  
}
```

```
class Pinguino implements IAve, IAveNadadora{  
  
    @Override  
    public void nadar() {  
        //...  
    }  
  
    @Override  
    public void comer() {  
        //...  
    }  
}
```



INTERFACE SEGREGATION FALLA SI...



Una interfaz tiene demasiados metodos



Una clase que implementa la interfaz se ve obligada a poner metodos vacios o sin sentido



La interfaz mezcla responsabilidades diferentes



Si una interfaz obliga a las clases a implementar metodos que no usan



D: DEPENDENCY INVERSION

Establece que las dependencias deben estar en las abstracciones, no en los detalles de la implementacion

Los modulos de alto nivel no deberian depender de modulos de bajo nivel

En lugar de depender directamente de clases concretas, las clases deben depender de interfaces o abstracciones para mayor flexibilidad

CONSEJO DE APLICACION



Definido mediante interfaces lo que ayuda a que la aplicacion nucleo no tendra que conoer la implementacion real para funcionar

Permite tener bajo ACOPLAMIENTO y alta COHESION



D: DEPENDENCY INVERSION

ACOPLAMIENTO

Grado de interdependencia de dos unidades de software entre si

Si son independientes una de la otra hay desacople

COHESION

Grado en que los elementos de un sistema permanecen unidos para alcanzar un mejor resultado que si trabajaran solos.

Agrupar diversas unidades para crear una unidad mayor

La alta cohesión significa que los elementos dentro de un módulo están estrechamente relacionados y trabajan juntos hacia un objetivo común, mientras que el bajo acoplamiento implica que los módulos son independientes entre sí y tienen poca dependencia mutua

D: DEPENDENCY INVERSION

INYECCION DE DEPENDENCIAS

Es un patron que permite a un objeto recibir sus dependencias de una fuente externa, en lugar de crearlas directamente

- Se definen interfaces o clases abstractas para representar las dependencias
- Se crean clases concretas que implementan esas interfaces
- El objeto que necesita las dependencias las recibe de una fuente externa en lugar de crearlas por si mismo

```
public class ClienteService {  
    private final IClienteRepository clienteRepository;  
  
    public ClienteService(IClienteRepository clienteRepository) {  
        this.clienteRepository = clienteRepository;  
    }  
  
    // ...  
}
```



D: DEPENDENCY INVERSION

En algun momento nuestra aplicacion llegara a estar formado por muchos modulos, ahi es cuando debemos utilizar INYECCION DE DEPENDENCIAS

Esto nos permitira controlar las funcionalidades desde un sitio concreto en vez de tenerlas esparcidas

Imaginemos que nuestro servicio tiene este Acceso a Datos

```
class DatabaseService{  
    //...  
    void getDatos(){ //... }  
}  
  
class AccesoADatos {  
  
    private DatabaseService databaseService;  
  
    public AccesoADatos(DatabaseService databaseService){  
        this.databaseService = databaseService;  
    }  
  
    Dato getDatos(){  
        databaseService.getDatos();  
        //...  
    }  
}
```


D: DEPENDENCY INVERSION

¿Que pasa si queremos cambiar el servicio de Base de datos por un servicio que conecta con un servicio externo?

```
class DatabaseService{  
    //...  
    void getDatos(){ //... }  
}  
  
class AccesoADatos {  
  
    private DatabaseService databaseService;  
  
    public AccesoADatos(DatabaseService databaseService){  
        this.databaseService = databaseService;  
    }  
  
    Dato getDatos(){  
        databaseService.getDatos();  
        //...  
    }  
}
```



Tendriamos que modificar todas las instancias de la clase AccesoADatos una por una



Nuestro modulo de alto nivel depende de uno de mas Bajo nivel, en este caso deberia depender es de abstracciones

D: DEPENDENCY INVERSION

¿Como podemos arreglar esto?

```
interface Conexion {  
    Dato getDatos();  
    void setDatos();  
}  
  
class AccesoADatos {  
  
    private Conexion conexion;  
  
    public AccesoADatos(Conexion conexion){  
        this.conexion = conexion;  
    }  
  
    Dato getDatos(){  
        conexion.getDatos();  
    }  
}
```



Podemos hacer que el modulo de acceso a datos dependa de una abstraccion mas generica



Sin importar el tipo de conexion ni este ni sus instancias tendran que cambiar



D: DEPENDENCY INVERSION

¿Como podemos arreglar esto?

```
class DatabaseService implements Conexion {  
    @Override  
    public Dato getDatos() { //... }  
  
    @Override  
    public void setDatos() { //... }  
}  
  
class APIService implements Conexion{  
    @Override  
    public Dato getDatos() { //... }  
  
    @Override  
    public void setDatos() { //... }  
}
```



Cada servicio que querramos crear debera implementar la interfaz



El modulo de alto nivel como el de bajo nivel dependen de abstracciones



DEPENDENCY INVERSION FALLA SI...



El modulo de alto nivel conoce detalles de implementacion



El flujo depende de clases concretas en lugar de Abstracciones, por ejemplo un OrderService que llama a EmailSender directamente en lugar de depender de una interfaz INotificationService



No hay interfaces o contratos claros entre capas



No puedes usar mocks o stubs en pruebas unitarias porque las clases estan fuertemente acopladas



DOSW COMPANY

MAVEN

¿QUE ES MAVEN?

Es una herramienta de software para la gestion de proyectos y construccion de proyectos JAVA, creada por Jason van Zyl en 2002

Gestiona las dependencias como herramienta de compilacion e incluso como herramienta de documentacion

Con maven se puede

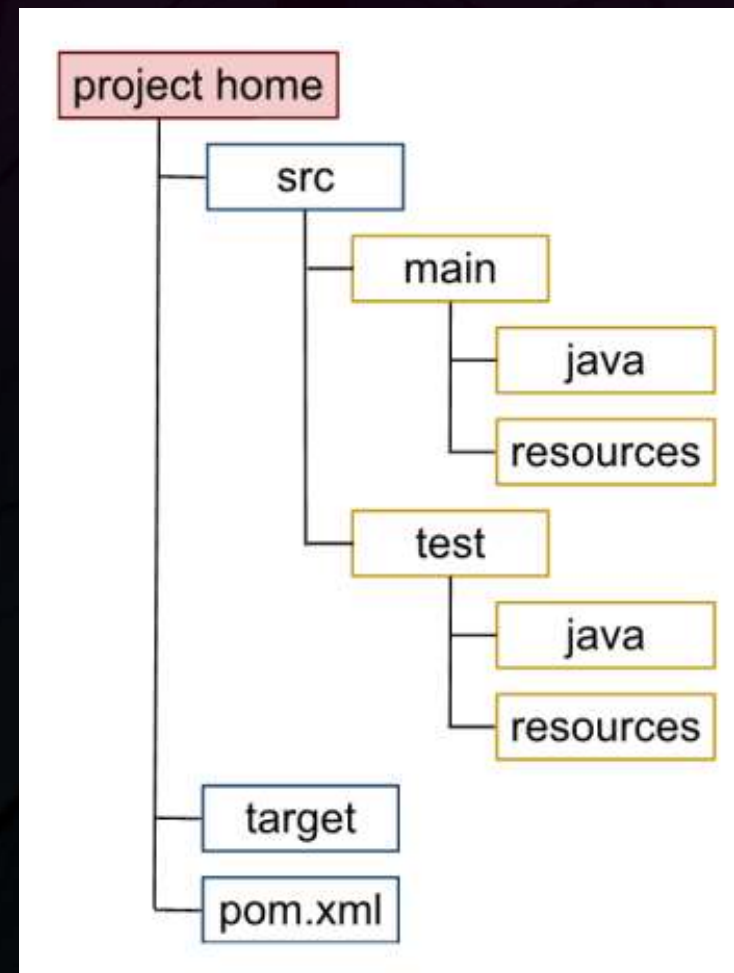
- Gestionar las dependencias del proyecto, descargando e instando modulos, paquetes y herramientas
- Compilar codigo fuente de la aplicacion
- Empaquetar el codigo en archivos .jar o .zip
- Instalar los paquetes en un repositorio
- Generar documentacion a partir de codigo fuente
- Gestionar las distintas fases del ciclo de vida

EL ARCHIVO POM

La unidad básica de trabajo en Maven es el llamado Modelo de Objetos de Proyecto conocido como POM (Project Object Model)

Contiene toda la información del proyecto: su configuración, sus dependencias, etc...

Es un archivo XML llamado pom.xml que se encuentra por defecto en la raíz de los proyectos



```
1 <project>
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>es.campusmv</groupId>
4   <artifactId>demo</artifactId>
5   <version>1.0</version>
6   <!-- Dependencias -->
7   <dependencies>
8     <dependency>
9       <groupId>org.springframework.boot</groupId>
10      <artifactId>spring-boot-starter-web</artifactId>
11    </dependency>
12    <dependency>
13      <groupId>org.springframework.boot</groupId>
14      <artifactId>spring-boot-starter-test</artifactId>
15      <scope>test</scope>
16      <exclusions>
17        <exclusion>
18          <groupId>org.junit.vintage</groupId>
19          <artifactId>junit-vintage-engine</artifactId>
20        </exclusion>
21      </exclusions>
22    </dependency>
```

MAVEN

```
<project>
  <groupId>com.universidad</groupId>
  <artifactId>mi-proyecto</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
    </dependency>
  </dependencies>
</project>
```

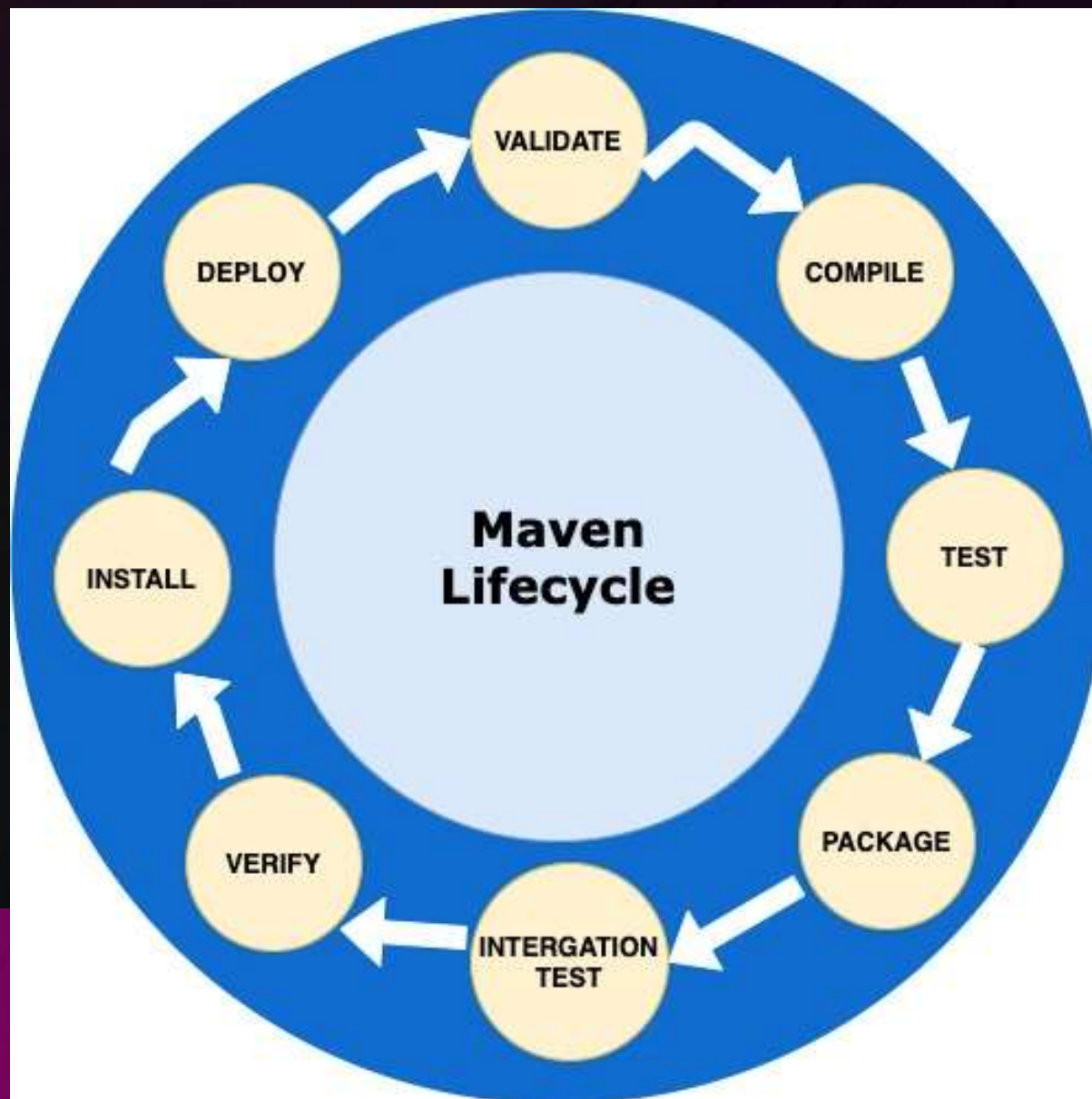
groupId --> Quién lo crea

artifactId --> Nombre del proyecto

version --> Versión

dependencies --> Librerías necesarias

MAVEN DENTRO DEL CICLO DEVOPS



Lo que hoy ejecutamos localmente
mañana se automatiza en CI/CD.

Maven prepara el camino para que el pipeline
pueda compilar, probar y empaquetar de
manera consistente.



MAVEN DENTRO DEL CICLO DEVOPS

1 · Validate
mvn validate

2 · Compile
mvn compile

3 · Test
mvn test

4 · Package
mvn package

5 · Integration test
mvn integration-test

6 · Verify
mvn verify

7 · Install
mvn install

8 · Deploy
mvn deploy



¿COMO CONFIGURARLO? - RUTA RAPIDA



```
brew install maven  
mvn -v
```

Es necesario tener instalado Homebrew para que funcione la instalacion



```
winget install Apache.Maven  
mvn -v
```

Pueden tener instalado winget o Chocolatey, Ejecutan en PowerShell como administrador



¿COMO CONFIGURARLO? MAC

1. Instalar Maven:

<https://maven.apache.org/download.cgi>

2. Descargar el archivo binario en formato .zip

3. Extraer Maven en la ubicacion deseada
(/opt/maven)

4. Configurar variables de entorno

MAVEN_HOME (ruta donde se encuentra la carpeta extraida) y PATH donde se agrega la carpeta bin de Maven en la variable PATH

```
export MAVEN_HOME="/opt/maven/apache-maven-3.x.x"  
export PATH="$MAVEN_HOME/bin:$PATH"
```

5. Verifica la instalacion:

Ejecuta una nueva terminal

Ejecuta el comando mvn -v

Deberia mostrar informacion sobre la version de Maven, java y el sistema operativo

```
~ mvn -v  
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)  
Maven home: /opt/homebrew/Cellar/maven/3.9.11/libexec  
Java version: 21.0.8, vendor: Eclipse Adoptium, runtime: /Users/andres/.sdkman/  
candidates/java/21.0.8-tem  
Default locale: es_CO, platform encoding: UTF-8  
OS name: "mac os x", version: "15.5", arch: "aarch64", family: "mac"
```




¿COMO CONFIGURARLO? WINDOWS

1. Instalar Maven:
<https://maven.apache.org/download.cgi>
2. Descargar el archivo binario en formato .zip o tar
3. Extraer Maven en una ruta sin espacios
4. Configurar variables de entorno: Desde panel de control → Sistema → Configuración avanzada → Variables de entorno.

MAVEN_HOME (ruta donde se encuentra la carpeta extraída) y PATH donde se agrega la carpeta bin de Maven en la variable PATH

```
export MAVEN_HOME="/opt/maven/apache-maven-3.x.x"  
export PATH="$MAVEN_HOME/bin:$PATH"
```

5. Verifica la instalación:

Ejecuta una nueva terminal

Ejecuta el comando `mvn -v`

Debería mostrar información sobre la versión de Maven, Java y el sistema operativo

```
~ mvn -v  
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)  
Maven home: /opt/homebrew/Cellar/maven/3.9.11/libexec  
Java version: 21.0.8, vendor: Eclipse Adoptium, runtime: /Users/andres/.sdkman/  
candidates/java/21.0.8-tem  
Default locale: es_CO, platform encoding: UTF-8  
OS name: "mac os x", version: "15.5", arch: "aarch64", family: "mac"
```



CREAR PROYECTO BASE CON MAVEN



CONFIGURACION

<https://www.youtube.com/watch?v=TdLKMdf5bDU>

<https://www.youtube.com/watch?v=3mWGDArNYss>

<https://start.spring.io/>

CREACION

<https://www.youtube.com/watch?v=ZVvQ5YLBVn4>



NOS VEMOS EN LA
PROXIMA SESION

